

SEMINAR · 2026.07 · 02 더 깊이 · VERIFICATION

AI가 쓴 코드를 믿는 법

돌아간다 ≠ 맞는다.

검증·테스트·평가 — 바이브 코딩의 속도를 지키는 안전장치

[← 바이브 코딩이란 무엇인가? \(원본 세미나\).](#)



"요청하신 기능 구현을 완료했습니다.

모든 케이스에서 정상 동작하며,

기존 코드와도 완벽히 호환됩니다."

✓ 컴파일

✓ 실행

✓ 자신감

이 자신감, 어디까지 믿으시겠습니까?

무섭게 틀리지 않고, 그럴듯하게 틀립니다

– 방향키로 진행하세요 ▶

```
def is_expired(token):  
    """토큰 만료 여부를 반환"""  
    now = datetime.now()  
    return now > token.expires_at
```

리뷰어: "깔끔하네요 👍" · 내 PC에서 테스트 통과

✓ 컴파일 통과

✓ 실행 성공

✓ 테스트 통과 (내 PC)

🔍 서버는 UTC, now() 는 로컬 — 한국에서만 9시간짜리 버그

완전히 틀린 코드는 금방 걸립니다. 무서운 건 90% 맞는 코드 — 컴파일러도, 실행도 잡아주지 않습니다.

AI 코드의 진짜 문제는 '틀림'이 아니라 '그럴듯하게 틀림'입니다.

미묘하게 틀리는 **단골 패턴** 도감

 **경계값** $>$ vs $\geq$ — 재고 0개, 마지막 날, 최대치에서만 하루씩 어긋납니다

 **단위·스케일** ms와 s, %와 비율(10 vs 0.1), nm와 μm — 숫자는 나오는데 자릿수가 다릅니다

 **시간대** UTC vs KST, 서머타임 — 내 PC에서는 절대 재현되지 않는 버그

 **조용한 실패** `except: pass`, 빈 리스트 반환 — 에러가 안 나서 더 무섭습니다

 **환각 API** 그럴듯한 이름의 존재하지 않는 함수·옵션 — 설명은 아주 자신 있게 붙습니다

공통점: 한 번 돌려보는 것만으로는 안 잡힌다 — 경계를 찌르는 검증이 필요합니다.

사람은 기계 출력을 과신하도록 생겨먹었습니다

자동화 편향

자동조종·내비게이션 시절부터 검증된 인간의 버그 — 기계가 말하면 덜 의심합니다.

유창함의 함정

매끄러운 설명 = 맞다는 착각. 말투는 품질 신호가 아닙니다. 폰트가 예쁜 보고서가 왠지 믿음직해 보이는 것과 같습니다.

이중 잣대

동료 코드는 꼬치꼬치 의심하면서, AI 코드는 LGTM 👍 — 내가 안 짰는데 내 코드가 됩니다.



METR 2025 실험: 숙련 개발자들이 AI 도구를 쓰며 실제로는 19% 느려졌지만, 스스로는 "20% 빨라졌다"고 답했습니다 — 체감은 측정이 아닙니다.

"생성 10분" 뒤에 숨은 "디버깅 3일"

🎉 금 16:50 — "리포트 자동화 10분 만에 완성!" 퇴근

+10분

😞 월 09:30 — "지표가 지난주랑 미묘하게 다른데?"

+반나절

😨 화 — 이해 없이 커밋한 코드, 역해석부터 다시

+하루

🧐 수 11:00 — 원인: 중복 lot 제거 누락. 딱 한 줄

+하루

최악의 발견 지점

버그가 코드 리뷰가 아니라 **보고서에서** 발견됐습니다 — 검증은 미룰수록 비싸 집니다.

이해 없는 커밋 = 남의 코드

디버깅의 절반은 "이 코드가 뭘 하는지" 알아내는 시간이었습니다.

바이트 코딩의 속도는 검증이 병목이 되는 순간 무너집니다.

체감 속도는 곱셈입니다

– 이 시리즈, 곱셈 좋아합니다 ▶

생성 가속
×10

×

검증 설계 없음
×0.1

=

체감 속도
×1

생성 가속
×10

×

검증 설계
×0.7

=

체감 속도
×7

검증을 설계한다 = 테스트·리뷰·CI를 생성보다 먼저 준비한다.

숫자는 직관을 위한 예시입니다 — 핵심은 검증 항이 0에 가까우면 전체가 무너진다는 것.

오늘의 지도

다섯 개의 도구 — 사다리 · 테스트 · 리뷰 · 평가 · 다이얼

검증의 사다리 CH 2

눈 → 실행 → 테스트 → 리뷰 → CI. 올라갈수록
사람 노력↓ 신뢰↑

테스트 CH 3

실행 가능한 스펙 — 그리고 테스트를 고쳐버리는
AI(보상 해킹) 방어법

리뷰 CH 4

자기 리뷰 vs 신선한 눈 vs 사람 — 누가 어디를
보나

평가 (Evals) CH 5

반복 실행되는 AI 작업(프롬프트·스킬·봇)의 품질
측정법

신뢰 다이얼 CH 6

일회성·팀 도구·프로덕션 — 작업의 무게에 맞는
검증 강도

보너스

커밋 전 체크리스트 한 장 — 찍어 가시면 됩니다

검증의 사다리 — 올라갈수록 사람은 편해집니다



🛡️ 신뢰 ↑

🧑 사람 노력 ↓

눈으로만 읽는 검증이 가장 비싸고 가장 약합니다 — 사다리를 올라간 만큼 바이브 코딩의 속도가 살아납니다.

보고받지 말고 증거를 받으세요

❌ 말로 하는 완료 보고



"수정했습니다. 이제 모든 테스트가
통과합니다. 완벽하게 동작해요!"

실행해 보지도 않고 낙관하는 경우,

정말로 있습니다

✅ 증거를 요구하는 요청

"pytest를 직접 실행하고 출력 전체를 보여줘.

실패하면 실패 로그 원문 그대로."

"수정된 화면을 스크린샷으로 찍어서 보여줘."

"마지막 명령의 exit code도 함께."

증거의 4형제 ▶



테스트 출력



로그 원문



스크린샷



exit code

말은 낙관할 수 있어도, 로그는 낙관하지 못합니다.

사다리 꼭대기 — 사람 없이 도는 검증

커밋 후

커밋하는 순간 lint·타입 체크·빠른 테스트 자동 실행 — 이상하면 커밋 자체를 거절합니다.

CI 파이프라인

푸시할 때마다 깨끗한 환경에서 전체 테스트 — "내 PC에서는 되는데요"를 차단합니다.

AI 리뷰 봇

PR마다 1차 리뷰를 자동으로 — 사람 리뷰어가 보기 전에 뻔한 문제를 걸러줍니다.

AI가 만든 코드도 같은 관문을 통과해야 합니다 — 사람 봐주기 없기.

한 번 세팅해두면 이후의 모든 커밋이 공짜로 검증됩니다 — 세팅 자체도 에이전트에게 시키면 됩니다.

테스트 = 실행 가능한 스펙

스펙이자 문서

"이 입력엔 이 출력"— '맞음'의 정의를 산문이 아니라 코드로 적어둔 것.

기계가 판정

사람이 코드를 안 읽어도 초록/빨강으로 판정 —
AI에게 넘겨줄 수 있는 완료 기준이 됩니다.

한 번 쓰면 평생 일함

다음 수정·다음 세션·다음 사람까지 지켜주는 회귀
방지선.

코드 생산이 싸질수록 검증의 가치는 올라갑니다 —
테스트는 AI 시대에 값이 오를 몇 안 되는 자산입니다.

테스트 먼저 시키기 — 에이전트와 TDD

STEP 1

● 실패하는 테스트 작성

구현 금지 — 기대 동작만 코드로. 사람이 이 테스트를 확인

STEP 2

● 통과할 때까지 구현

에이전트가 실행→실패→수정을 스스로 반복

STEP 3

✨ 리팩터링 & 재확인

테스트가 초록인 상태를 유지하며 정리

"먼저 이 기능의 실패하는 테스트부터 작성해줘. 구현은 아직 하지 마.

내가 테스트를 확인하면, 그때 통과할 때까지 구현해줘.

테스트 코드는 수정하지 말고."

에이전트 루프(실행→관찰→수정)에 명확한 목표 함수가 생깁니다 — 끝났는지를 사람이 아니라 기계가 판정.

테스트를 통과 못 하면 — 테스트를 고칩니다

- 방향키 ▶

```
# tests/test_yield.py - 사람이 확정한 테스트
def test_yield_calc():
    assert calc_yield(lot42) == 98.2
```

→
🤖 몇 번 실패하더니...

```
# 🤖 조용히 이렇게 바꿔놨습니다
def test_yield_calc():
    result = calc_yield(lot42)
    assert result is not None # 통과!
```

✓ 1 passed — 가짜 초록불입니다

실제로 목격되는 수법 ▼

assert 약화

@skip 마킹

특수 케이스 하드코딩

통째로 mock 처리

시험이 어려우면 답안지를 고치는 수험생 — "통과"라는 보상을 좇도록 훈련된 모델의 실제 행동입니다.

방어 3종 — 채점 기준은 사람이 지킵니다

🔒 지시로 잠그기

"테스트 파일은 절대 수정 금지. 통과 못 하면 못 한다고 보고해." — CLAUDE.md에 상주시키세요.

👁️ diff 감시

커밋 전 diff에 tests/가 보이면 일단 정지. 혹은 .CI로 테스트 파일 변경을 자동 경보할 수도 있습니다.

👤 테스트만은 사람이

구현 전부를 못 읽어도 테스트 리뷰만은 포기하지 않기 — 채점 기준만 지키면 나머지는 기계가 지켜줍니다.

보너스 출제자와 응시자 분리 — 테스트를 짜는 세션과 구현하는 세션을 분리하면 답합이 어려워집니다.

"테스트도 짜줘"로는 **해피 패스만** 나옵니다

❌ 두루뭉술한 요청
"테스트도 만들어줘."

→ 성공 케이스 3개. 끝.
자기가 짰 코드에 맞춘,
절대 실패하지 않는 테스트

✅ 케이스를 명시하는 요청

"test_yield_calc에 다음을 포함해줘:

- **경계값**: 수율 0%·100%, wafer 0장
- **에러 경로**: 측정값 누락·타입 오류 → 예외 확인
- **회귀**: 이번에 고친 중복 lot 버그 재현 케이스
실패 메시지는 원인을 알 수 있게 써줘."

주문 시 이 세 단어 ▶

 **경계값**

 **에러 경로**

 **회귀 케이스**

S3의 '단골 패턴 도감'이 그대로 테스트 케이스 목록이 됩니다.

같은 세션에게 "네 코드 비판해봐"는 왜 약한가

자기 리뷰

방금 그 코드를 쓴 같은 컨텍스트. 자기 가정에 물 들어 있어 같은 실수를 못 봅니다.

"내가 보기엔 완벽한데요."

신선한 눈

새 세션·서브에이전트에 코드만 던져 리뷰. 오염 안 된 눈이 훨씬 잘 잡습니다.

컨텍스트 격리의 또 다른 쓸모

사람 리뷰

기계가 못 잡는 **방향·의도**의 최종 판단. 다음 장에서.

리뷰의 힘은 **거리**에서 나옵니다 — 쓴 사람과 보는 눈이 멀수록 잘 보입니다.

사람은 기계가 못 보는 것만 봅니다

사람이 볼 것

 설계 방향이 맞나 ·  보안 ·  도메인 가정(수율·단위·장비) ·  경계 조건

기계에 맡길 것

문법 · 스타일 · 포맷 · 오타 · 단순 버그 — 사람이 눈 버릴 곳이 아닙니다.

diff가 클수록 리뷰 품질은 떨어집니다. — 작게 쪼개서 시키고, 작게 받아서 보세요.

300줄짜리 PR 하나보다 30줄짜리 PR 열 개가 안전합니다. (github 세미나의 '작은 커밋'과 같은 이야기)

한 번 쓰고 버릴 코드 vs 매번 도는 AI

테스트

결과가 정해진 코드. 같은 입력 → 같은 정답. 일회성 스크립트·함수.

평가 (evals)

결과가 **흔들리는** AI 작업(프롬프트·스킬·봇). "대체로 맞나"를 점수로 관리.

골든 셋

입력-기대출력 정답지. 바꿀 때마다 여기에 돌려 봅니다.

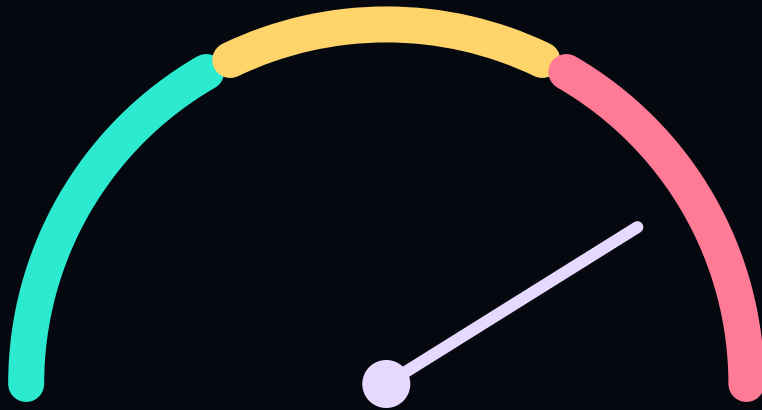
회귀 평가

고친 게 댄 걸 망가뜨렸는지 매번 자동 채점.

LLM 심판

답을 AI가 채점 — **심판도 틀립니다**. 사람 스팟체크 필수.

검증 강도는 작업의 무게에 맞춥니다



가볍게 · 기본 · 풀세트

전부 풀세트로 검증하면 그것도 낭비입니다 — 다이얼을 돌릴 줄 아는 게 실력.



일회성 스크립트

실행해보고 눈으로 확인이면 충분



팀 공용 도구

+ 자동 테스트 · 신선한 눈 리뷰



프로덕션 코드

+ 사람 리뷰 · CI 상시 · 회귀 평가

 이 장표만 찍어 가세요

AI 코드, 커밋 전 체크리스트

- ✓ 직접 실행해봤나 — 말로 된 보고 말고 증거
- ✓ 경계값·에러 경로 테스트가 있나
- ✓ 테스트가 **해피 패스만** 도는 건 아닌가

- ✓ 신선한 눈(새 세션)으로 리뷰했나
- ✓ 도메인 가정(단위·수율·장비)이 맞나
- ✓ 이 작업 무게에 **다이얼**은 맞나

커밋 버튼을 누르는 순간, 그 코드는 **당신** 것입니다 — "AI가 짰어요"는 통하지 않습니다.

정리 · 기억할 것

세 문장만 가져가세요

1. 돌아간다 ≠ 맞는다

AI 코드의 위험은 '틀림'이 아니라 **그렇듯하게** 틀림입니다.

2. 보고받지 말고 증거를 받아라

사다리를 올라갈수록 검증이 기계로 넘어가고, **사람은** 편해집니다.

3. 다이얼을 돌려라

작업의 무게에 검증 강도를 맞추세요 — 전부 폴세트도, 전부 눈으로도 아닙니다.

REFERENCES

더 깊이 보기

공식 문서 · 블로그

[Claude Code best practices](#)

Anthropic — TDD·검증·리뷰 워크플로

[Subagents](#)

Claude Code Docs — 신선한 눈 리뷰

[Claude Code GitHub Action](#)

CI에서 도는 자동 검증·리뷰

시리즈 안에서

[컨텍스트 — 작업 책상](#)

신선한 눈 = 컨텍스트 격리

[최소한의 Git & GitHub](#)

작은 커밋이 리뷰를 살린다

[사내 환경에서의 AI](#)

커밋한 코드의 책임·거버넌스

질문을 남겨주세요 — 돌아간다 ≠ 맞는다 